Task 4: Attendance Anomaly Detection

Documentation | Isolation Forest, model version attendance-anomaly-1.0.0 | Prepared by Katarukonda Ashok (GM00154) | October 2026

1. What this task is, in plain words

HR needs to see unusual attendance quickly: someone absent for two weeks without a note, an impossible 19-hour average day, or a sudden break from a person's normal pattern. There are no labels saying which months are "bad", so this is **unsupervised** learning: the model learns what normal looks like and scores how far each month is from it.

The problem with no labels is that we cannot tell whether the model is any good. So we **plant known anomalies** in a copy of the data, remember exactly which rows we changed, and measure how many the model finds (**catch rate**) and how many normal rows it wrongly flags (**false-alarm rate**).

2. Data and features

The attendance table has **7,339 employee-months** for 528 employees (Apr 2025 to Sep 2026), with no missing values. The feature pipeline (`src/features/attendance.py`, same fit/transform style as Task 2) keeps eight raw measures and adds six engineered ones:

Feature	Meaning
attendance_rate, unpaid_absence_days, longest_unpaid_streak_days, late_arrivals, avg_hours_worked, overtime_hours, half_days_unpaid, paid_leave_days	Taken as they are
present_ratio, wfh_share	Present days / scheduled days; work-from-home days / present days
rate_change_vs_prev, hours_change_vs_prev	Change from the person's previous month
rate_vs_own_mean, hours_vs_own_mean	Distance from **this person's own** usual level (learned in fit). This is what detects a sudden pattern change

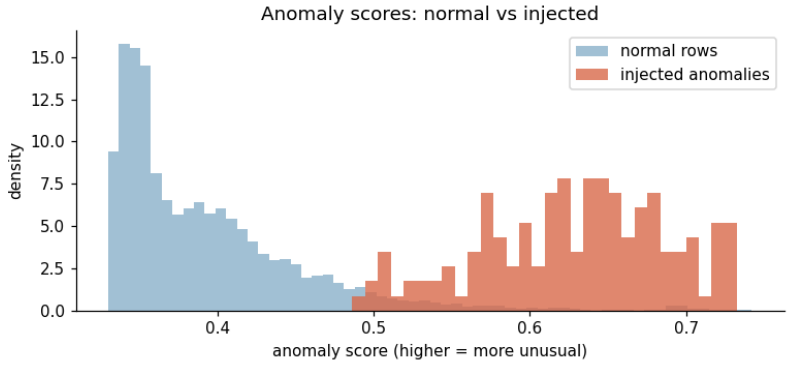
3. Creating ground truth: the injection script

`src/anomaly/inject.py` is repeatable with a fixed seed (42): running it twice gives identical rows (checked: True). The 150 rows labelled in the workbook are set aside first, leaving 7,189 rows treated as normal; then **140** anomalies are injected into healthy months and logged (`outputs/tables/injected_rows_log.csv` records the id, type, changed columns, old and new values).

Injected type	Rows	What changes
LONG_ABSENCE	50	7-15 unpaid days in a row; present days and attendance rate fall to match
IMPOSSIBLE_HOURS	45	Average hours per day set to 13.5-21
PATTERN_CHANGE	45	Attendance falls to 50-75%, late arrivals jump, hours dip

4. How Isolation Forest works

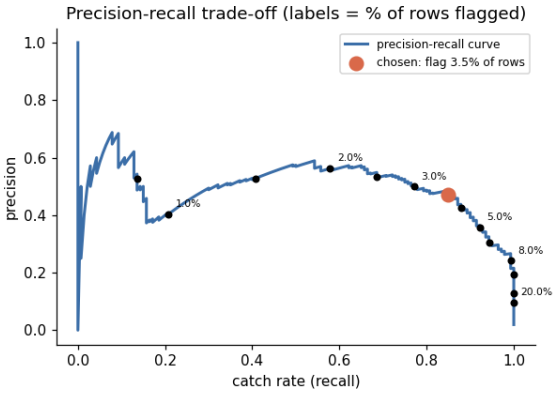
It builds many random trees. Each tree splits the data on a random feature at a random value. A strange month is separated from the others after only a few splits (a **short path**); a normal month is buried in the crowd and needs many splits. The average path length becomes the **anomaly score**: shorter path, higher score. The model (300 trees, seed 42) never sees which rows were injected.

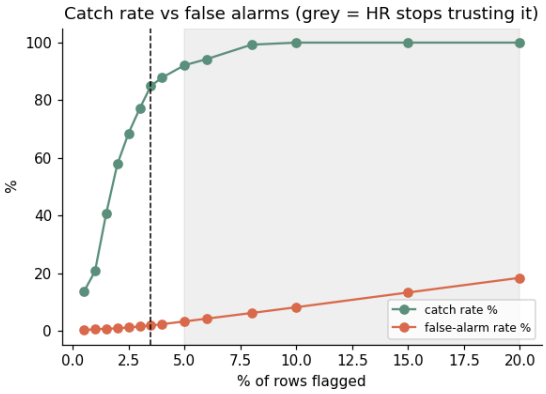


5. Threshold tuning

For each share of rows flagged (the "contamination" idea) we recorded how many injected anomalies were caught and how many normal rows were wrongly flagged:

Rows flagged	Threshold (score)	Caught	Missed	False alarms	Catch rate	False-alarm rate	Precision
1.0%	0.680	29	111	43	21%	0.6%	40%
2.0%	0.621	81	59	63	58%	0.9%	56%
3.0%	0.578	108	32	108	77%	1.5%	50%
3.5%	0.566	119	21	133	85%	1.9%	47%
4.0%	0.552	123	17	165	88%	2.3%	43%
5.0%	0.531	129	11	231	92%	3.3%	36%
10.0%	0.479	140	0	579	100%	8.2%	19%
20.0%	0.436	140	0	1298	100%	18.4%	10%





6. Chosen threshold and the reason

Flag the top 3.5% most unusual months (anomaly score at least 0.566). This catches **85%** of injected anomalies (119 of 140), wrongly flags only **1.9%** of normal rows, and has precision 47%. That is about **14 employee-months per calendar month** for HR to review.

Why: rule = best F1 among thresholds that flag at most 5% of rows. Below about 3% the catch rate falls steeply; above about 4% each extra anomaly costs many extra false alarms and precision falls toward 35% and below. The hint warns that flagging 20% of rows makes HR stop trusting the list, and the right-hand chart shows why: the false-alarm line stays low up to about 5%, while going to 10% would catch every injected anomaly but flood HR with wrong flags.

7. What it catches and what it misses

Type	Share flagged at the chosen threshold
LONG_ABSENCE	100%
PATTERN_CHANGE	100%
IMPOSSIBLE_HOURS	53%
normal rows (false alarms)	1.9%

Anomalies that move several features at once (absences, pattern changes) are found every time. A single extreme column (impossible hours) is diluted among 14 features, so only about half are caught. The fix is a plain rule: **average hours above 12** catches 45 of 45 impossible-hours rows with 0 false alarms (the highest normal month is 11.1 hours). A good production design is **hard rules for impossible values, Isolation Forest for subtle patterns**.

8. Second exam: the labels shipped with the dataset

The workbook has 150 labelled anomalies (absence streaks, impossible hours, late spikes, sudden drops, zero attendance). They were **set aside before training**, so the model has never seen them. Scoring the full original table with the saved model and threshold flags 238 months (3.2%): **104 of the 150 caught (69%)**, precision 44%.

Provided type	Share flagged
ABSENCE_STREAK	100%
IMPOSSIBLE_HOURS	46%
LATE_SPIKE	10%

Provided type	Share flagged
SUDDEN_DROP	100%
ZERO_ATTENDANCE	100%

The dataset authors' reference Isolation Forest (3% contamination) reports precision 0.40 and recall 0.59; this model is comparable or slightly better. The exam also exposed a blind spot the first test could not: **late-arrival spikes** (13+ late arrivals in a month) are mostly missed, because the injection design had no such type. That is a reason to add a late-count rule and a LATE_SPIKE injection in the next iteration.

9. Saved model and how to run it

Saved with `joblib`: the full pipeline, the threshold and a metadata file (version, seed, features, threshold rule, catch and false-alarm rates). Reloading gives identical scores (True).

```
jupyter nbconvert --to notebook --execute --inplace Task4_Anomaly_Detection/task4_anomaly_detectio
n.ipynb
```

```
from src.anomaly.detect import load_detector, anomaly_scores
pipe, threshold, meta = load_detector('Task4_Anomaly_Detection/outputs/artifacts/attendance-anomal
y-1.0.0.joblib')
flags = anomaly_scores(pipe, new_month_rows) >= threshold
```

10. Limitations (say these in the review)

- Injected anomalies are synthetic and fairly obvious, so the first catch rate is optimistic; the provided-label exam is the more realistic number.
- Single-feature anomalies (hours, late spikes) are weak spots and need hard rules.
- Some "false alarms" may be real, unlabelled oddities in the data.
- The threshold is a flag rate. Revisit it when HR gives feedback on the flagged list.

11. Completion criteria

Completion criterion	Status	Evidence
Injection script repeatable with a fixed seed	Done	src/anomaly/inject.py, seed 42; test and notebook confirm identical reruns
Catch rate and false-alarm rate reported	Done	Section 5 table; chosen point 85% caught, 1.9% false alarms
Threshold chosen with a written reason	Done	Section 6
Model saved with metadata	Done	outputs/artifacts/attendance-anomaly-1.0.0.joblib and .meta.json

Next: Phase 2 applies the same thinking to payroll, then wraps all models in a secure, tested service.